

3130. Find All Possible Stable Binary Arrays II

Solve

Hard

Topics

Companies

Hint

You are given 3 positive integers `zero`, `one`, and `limit`.

A **binary array** `arr` is called **stable** if:

- The number of occurrences of 0 in `arr` is **exactly** `zero`.
- The number of occurrences of 1 in `arr` is **exactly** `one`.
- Each **subarray** of `arr` with a size greater than `limit` must contain **both** 0 and 1.

Return the *total* number of **stable** binary arrays.

Since the answer may be very large, return it **modulo** $10^9 + 7$.

Example 1:

Input: zero = 1, one = 1, limit = 2

Output: 2

Explanation:

The two possible stable binary arrays are `[1,0]` and `[0,1]`.

Example 2:

Input: zero = 1, one = 2, limit = 1

Output: 1

Explanation:

The only possible stable binary array is `[1,0,1]`.

Example 3:

Input: zero = 3, one = 3, limit = 2

Output: 14

Explanation:

All the possible stable binary arrays are `[0,0,1,0,1,1]`, `[0,0,1,1,0,1]`, `[0,1,0,0,1,1]`, `[0,1,0,1,0,1]`, `[0,1,0,1,1,0]`, `[0,1,1,0,0,1]`, `[0,1,1,0,1,0]`, `[1,0,0,1,0,1]`, `[1,0,0,1,1,0]`, `[1,0,1,0,0,1]`, `[1,0,1,0,1,0]`, `[1,0,1,1,0,0]`, `[1,1,0,0,1,0]`, and `[1,1,0,1,0,0]`.

Constraints:

- `1 <= zero, one, limit <= 1000`

Python:

```
MOD = 1_000_000_007
MAXN = 1000
fact = [0] * (MAXN + 1)
invfact = [0] * (MAXN + 1)
```

```
def init():
    fact[0] = 1
    for i in range(1, MAXN + 1):
        fact[i] = (fact[i - 1] * i) % MOD
    invfact[MAXN] = pow(fact[MAXN], MOD - 2, MOD)
    for i in range(MAXN, 0, -1):
        invfact[i - 1] = (invfact[i] * i) % MOD
```

```
init()
```

```
class Solution:
```

```
    def numberOfStableArrays(self, zero: int, one: int, limit: int) -> int:
```

```
        if zero > one:
            zero, one = one, zero
```

```
        if limit == 1:
            if zero == one: return 2
            if zero + 1 == one: return 1
            return 0
```

```
        def ncr(n: int, r: int) -> int:
            return fact[n] * invfact[r] * invfact[n - r]
```

```
        def ways(n: int, k: int) -> int:
            j, total, flag, remaining = 0, 0, True, n
            while j <= k <= remaining:
                term = ncr(k, j) * ncr(remaining - 1, k - 1)
                total = total + term if flag else total - term
                j += 1
                flag = not flag
                remaining -= limit
            return total
```

```
        result = 0
        start = (zero + limit - 1) // limit
        prv, cur, nxt = 0, ways(one, start), ways(one, start + 1)
```

```

for k in range(start, zero + 1):
    result += (prv + 2 * cur + nxt) * ways(zero, k)
    prv, cur, nxt = cur, nxt, ways(one, k + 2)

```

```

return result % MOD

```

JavaScript:

```

var numberOfStableArrays = function(zero, one, limit) {
    const mod = 1000000007,
        limit1 = limit + 1
    var dp0 = Array(one + 1).fill(0),
        dp1 = Array.from({ length: zero + 1 }, () => Array.from({ length: one + 1 }, () => 0)),
        i, j
    for (j = 0; j <= Math.min(one, limit); j++)
        dp1[0][j] = 1
    for (i = 1; i <= zero; i++) {
        dp0[0] = (i <= zero && i <= limit) ? 1 : 0
        for (j = 1; j <= one; j++) {
            dp0[j] = (dp0[j] + dp1[i - 1][j]) % mod
            if (i > limit)
                dp0[j] = (dp0[j] - dp1[i - limit1][j] + mod) % mod
            dp1[i][j] = (dp1[i][j] - 1 + dp0[j - 1]) % mod
            if (j > limit)
                dp1[i][j] = (dp1[i][j] - dp0[j - limit1] + mod) % mod
        }
    }
    return (dp0[one] + dp1[zero][one]) % mod
};

```

Java:

```

class Solution {
    public int numberOfStableArrays(int zero, int one, int limit) {
        int MOD = (int)1e9 + 7;

        int[][] dp0 = new int[one + 1][zero + 1];
        int[][] dp1 = new int[one + 1][zero + 1];

        for (int i = 1; i <= Math.min(one, limit); i++) {
            dp1[i][0] = 1;
        }

        for (int j = 1; j <= Math.min(zero, limit); j++) {
            dp0[0][j] = 1;
        }
    }
}

```

```

for (int o = 1; o <= one; o++) {
    for (int z = 1; z <= zero; z++) {
        if (o == 0 && z == 0) continue;

        dp0[o][z] = dp1[o][z - 1];
        dp0[o][z] = (dp0[o][z] + dp0[o][z - 1]) % MOD;
        if (z - limit - 1 >= 0) {
            dp0[o][z] = (dp0[o][z] - dp1[o][z - limit - 1] + MOD) % MOD;
        }

        dp1[o][z] = dp0[o - 1][z];
        dp1[o][z] = (dp1[o][z] + dp1[o - 1][z]) % MOD;
        if (o - limit - 1 >= 0) {
            dp1[o][z] = (dp1[o][z] - dp0[o - limit - 1][z] + MOD) % MOD;
        }
    }
}

return (dp0[one][zero] + dp1[one][zero]) % MOD;
}

```